

Chapter 6 – OpenID Connect (OIDC)

Learning Objectives

After completing this chapter, you should be able to:

- Understand why OpenID Connect (OIDC) was created.
 - Explain the relationship between OAuth 2.0 and OIDC.
 - Understand ID Tokens and JWTs.
 - Explain OIDC authentication flow.
 - Compare OIDC with SAML.
 - Understand where OIDC is used in Google Workspace and modern applications.
 - Troubleshoot common OIDC issues.
-

1. Business Problem

In the previous chapter, we learned that OAuth authorizes applications.

But imagine an application asks Google:

"Can you tell me who this user is?"

OAuth cannot reliably answer that question.

OAuth only says:

"This application has permission to access these resources."

It does **not** identify the user.

This is why OpenID Connect was created.

2. What is OpenID Connect?

OpenID Connect (OIDC) is an authentication layer built on top of OAuth 2.0.

It allows applications to verify the identity of a user.

In simple terms:

- OAuth answers: **What can the application access?**
 - OIDC answers: **Who is the user?**
-

Real Enterprise Example

A company has an internal HR portal.

The HR portal needs to:

- Know which employee signed in.
- Display the employee's name.
- Display the employee's department.

The HR portal does **not** need to read Gmail or Drive.

OIDC allows the application to authenticate the user and receive identity information securely.

3. OAuth vs OIDC

OAuth 2.0	OpenID Connect
Authorization	Authentication
Access Tokens	ID Tokens
API permissions	User identity
Resource access	Login

Think of OIDC as an extension of OAuth rather than a replacement.

4. OIDC Components

Component	Purpose
User	Person signing in
Client	Application requesting login
Authorization Server	Google Identity

Component	Purpose
Resource Server	Google APIs (if needed)
ID Token	User identity
Access Token	API access

5. OIDC Login Flow

```
User
↓
Application
↓
Redirect to Google Login
↓
User authenticates
↓
Google requests consent (if required)
↓
Google issues
- ID Token
- Access Token (optional, depending on scopes)
↓
Application validates the ID Token
↓
Application creates a user session
↓
```

User is logged in

Notice the difference:

The application checks the **ID Token** to confirm the user's identity.

6. What is an ID Token?

An ID Token is a security token that contains information about the authenticated user.

It is intended for the application, not for calling Google APIs.

Typical information includes:

- User ID
- Email address
- Name
- Authentication time
- Token expiration
- Issuer
- Audience

The ID Token is digitally signed by Google.

Example ID Token Claims

```
iss: https://accounts.google.com
sub: 10987654321
email: rahul@company.com
email_verified: true
name: Rahul
aud: hr-portal-client-id
exp: 1734529200
```

```
iat: 1734525600
```

These fields are called **claims**.

7. JWT (JSON Web Token)

Most ID Tokens are formatted as **JWTs (JSON Web Tokens)**.

A JWT has three parts:

```
Header
.
Payload
.
Signature
```

Example:

```
xxxxx.yyyyy.zzzzz
```

Each section is Base64URL encoded.

JWT Structure

```
Header
↓
Algorithm
Token Type
↓
Payload
```

↓
Claims
↓
Signature
↓
Verification

8. JWT Header

The header tells the application:

- Which signing algorithm was used
- The token type

Example:

```
{  
  "alg": "RS256",  
  "typ": "JWT"  
}
```

9. JWT Payload

The payload contains claims.

Examples:

- Email
- Name
- User ID
- Expiration
- Issued At
- Audience

Important:

The payload is **encoded**, not **encrypted**.

Anyone with the token can decode it.

Sensitive data should **not** be stored in the payload.

10. JWT Signature

The signature protects the token from modification.

Google signs the token using its private key.

Applications verify the signature using Google's public keys.

If the payload is modified,

the signature becomes invalid,

and the token is rejected.

JWT Verification Flow

Google

↓

Create JWT

↓

Sign with Private Key

↓

Application

↓

Verify Signature using Google's Public Key

↓

Signature Valid?

↓

Yes

↓

Accept Token

11. Common OIDC Claims

Claim	Meaning
iss	Issuer of the token
sub	Unique user identifier
aud	Intended audience (Client ID)
exp	Expiration time
iat	Issued at time
email	User email address
email_verified	Whether the email is verified
name	Display name

Applications should validate these claims before trusting the token.

12. OIDC vs SAML

SAML	OIDC
XML-based	JSON-based
Assertions	ID Tokens
Enterprise SSO	Modern web/mobile apps
More verbose	Lightweight
Common in traditional enterprise apps	Common in cloud-native applications

13. OIDC in Google Workspace

Examples where OIDC is commonly used:

- Internal web applications
- Mobile applications
- SaaS platforms
- Cloud-native applications
- Customer portals

Developers often choose OIDC because it is easier to integrate with modern applications.

14. Security Best Practices

When validating an ID Token:

- Verify the signature.
- Verify the issuer (`iss`).
- Verify the audience (`aud`).
- Check the expiration (`exp`).
- Confirm the token has not been tampered with.

Never trust an ID Token without validation.

15. Common OIDC Errors

Invalid Audience

Cause:

The token was issued for a different application.

Solution:

Verify the application's Client ID matches the `aud` claim.

Expired Token

Cause:

The ID Token has expired.

Solution:

Authenticate again to obtain a new token.

Invalid Signature

Cause:

The token has been modified or was not signed by Google.

Solution:

Reject the token.

Incorrect Issuer

Cause:

The application expected Google's issuer but received a token from another provider.

Solution:

Verify the `iss` claim.

16. Troubleshooting

User cannot sign in to an application using OIDC.

Questions to ask:

1. Did the user authenticate successfully?
2. Was an ID Token issued?
3. Is the signature valid?
4. Has the token expired?

5. Does the `aud` claim match the application's Client ID?
 6. Is the `iss` claim trusted?
 7. Are the required scopes configured (for any API access)?
-

Enterprise Scenario

A company builds an internal leave management application.

Requirements:

- Authenticate employees with Google.
- Display employee names and email addresses.
- Do not request access to Gmail or Drive.

Solution:

Use OIDC.

The application receives an ID Token,

verifies it,

creates a session,

and displays the employee's identity.

No unnecessary API permissions are requested.

Best Practices

- ✓ Use OIDC for authentication in modern applications.
- ✓ Validate every ID Token.
- ✓ Do not store sensitive information in JWT payloads.
- ✓ Always verify signatures.
- ✓ Check expiration times.

- ✓ Keep authentication separate from authorization.
-

Common Mistakes

- ✗ Confusing ID Tokens with Access Tokens.
 - ✗ Using an ID Token to call Google APIs.
 - ✗ Skipping signature verification.
 - ✗ Ignoring the `aud` or `iss` claims.
 - ✗ Assuming a decoded JWT is automatically trustworthy.
-

Interview Questions

1. What is OpenID Connect?
 2. How does OIDC differ from OAuth 2.0?
 3. What is an ID Token?
 4. What is a JWT?
 5. What information is stored in an ID Token?
 6. Why is the JWT signature important?
 7. How would you troubleshoot an invalid ID Token?
 8. When would you choose OIDC over SAML?
-

Key Takeaways

OAuth provides **authorization**.

OIDC provides **authentication**.

ID Tokens identify users.

Access Tokens authorize API access.

JWTs carry identity information securely.

Always validate tokens before trusting them.

Chapter Summary

In this chapter you learned:

- ✓ Why OIDC exists.
- ✓ How OIDC builds on OAuth 2.0.
- ✓ The OIDC authentication flow.
- ✓ ID Tokens.
- ✓ JWT structure and claims.
- ✓ OIDC vs SAML.
- ✓ OIDC security best practices.
- ✓ Common troubleshooting scenarios.

The next chapter will move from identity and authentication into **SCIM (System for Cross-domain Identity Management)**, where you'll learn how enterprises automatically provision, update, suspend, and deprovision users between identity providers and Google Workspace.